MCP–RPC Bridge

Framework & Technical Design Document

Version **1.0** · 2026-03-30

Status: **DRAFT**

Expose gRPC/Protobuf services as MCP tools — high-performance, fully decoupled, minimal footprint

1. Overview

This document describes the design of the MCP–RPC Bridge: a thin adapter layer that automatically exposes Protobuf-defined RPC services as Model Context Protocol (MCP) tools, allowing LLMs to invoke backend services without any per-service integration code.

1.1 Goals

- Zero per-method boilerplate — register once at the Service level; every method is exposed automatically.
- High performance — lazy schema generation, zero-copy deserialization on the call hot path.
- Full decoupling — the MCP layer is entirely transport-agnostic (gRPC, in-proc, Thrift, custom).
- Small footprint — target < 1 000 lines of production code excluding tests.
- Robust to LLM quirks — graceful handling of unknown fields, streaming RPCs, and long tool names.

1.2 Out of scope

- Authentication and per-tool authorization (delegated to the RPC layer).
- LLM-to-MCP client implementation (this document covers the server / adapter side only).
- Dynamic proto loading at runtime (descriptors assumed to be compiled in at build time).

1.3 Terminology

Term	Meaning
MCP	Model Context Protocol — JSON-RPC 2.0 dialect for exposing tools to LLMs
Tool	A named, schema-described callable in MCP (maps 1-to-1 to an RPC method)
ToolEntry	In-memory record: tool name, description, MethodDescriptor pointer, and lazy JSON Schema
RpcInvoker	Pure interface abstracting the underlying transport (gRPC, in-proc, etc.)
Descriptor	Protobuf FileDescriptorProto / MethodDescriptor available at runtime

2. Architecture

The bridge is split into two layers separated by a single narrow interface (RpcInvoker). Neither layer imports internals of the other.

2.1 Layer map

Layer	Components	Responsibility
MCP Adapter	MCPSession, ToolRegistry, ProtoSchemaConverter, ArgumentsDeserializer	Protocol handling, schema generation, argument conversion
Transport boundary	RpcInvoker (interface)	Single crossing point — serialized bytes in, serialized bytes out
RPC Layer	GrpcInvoker, InProcInvoker (impls)	Transport-specific invocation; never imported by MCP layer

2.2 Component responsibilities

MCPSession

- Implements the JSON-RPC 2.0 state machine (initialize → tools/list → tools/call).
- Routes requests to ToolRegistry (list) or ArgumentsDeserializer + RpcInvoker (call).
- Contains no business logic — pure protocol and dispatch.

ToolRegistry

- An immutable flat_hash_map built once at startup: tool name → ToolEntry.
- ToolEntry holds only a name string, a description string, and a const MethodDescriptor pointer.
- JSON Schema is a — not computed until the first call.

ProtoSchemaConverter

- Stateless recursive function: DescriptorProto → JSON Schema object.
- Nested message types are placed in $defs and referenced by $ref — avoids duplication and cycles.
- Runs once per message type; result is cached inside ToolEntry.

ArgumentsDeserializer

- Uses directly — no hand-written field traversal.
- Accepts a into the original network buffer — zero copy on the hot path.
- to tolerate occasional LLM hallucinated keys.

RpcInvoker interface

```
class RpcInvoker {
public:
    virtual ~RpcInvoker() = default;

    // method_full_name: "package.ServiceName/MethodName"
    virtual std::future<absl::StatusOr<std::vector<uint8_t>>> Invoke(
        std::string_view         method_full_name,
        std::span<const uint8_t> request_bytes
    ) = 0;
};
```

> *This is the ONLY symbol that crosses the MCP ↔ RPC boundary. Concrete implementations (GrpcInvoker, InProcInvoker) are injected at construction time and never imported by the adapter.*

3. Request Data Flow

3.1 tools/list

```
LLM              MCPSession       ToolRegistry     ProtoSchemaConverter
 |                   |                 |                     |
 |-- tools/list ----->|                 |                     |
 |                   |-- list() ------->|                     |
 |                   |                 | GetInputSchema()    |
 |                   |                 |-- (LazyValue) ---->|
 |                   |                 |<-- JSON Schema ----+
 |<-- [{name,desc,    |<-- [ToolEntry] -+                     |
 |     inputSchema}]  |                 |                     |
```

LazyValue ensures schema generation runs at most once per message type, even under concurrent requests (backed by std::call_once).

3.2 tools/call

```
LLM            MCPSession      ArgDeserializer    RpcInvoker
 |                |                 |                 |
 |-- tools/call --->|                 |                 |
 |  {name, args}   |-- Deserialize -->|                 |
 |                 |  string_view     |                 |
 |                 |<-- proto bytes --+                 |
 |                 |---------- Invoke() -------------->|
 |                 |<--------- response bytes ---------+
 |<-- result ------ | (JSON via JsonMessageToString)    |
```

4. ProtoSchemaConverter

4.1 Type mapping rules

Protobuf type	JSON Schema mapping	Notes
TYPE_STRING	{"type":"string"}	—
TYPE_BYTES	{"type":"string","contentEncoding":"base64"}	Semantic loss; annotated in description
TYPE_INT32 / INT64 / UINT32 / UINT64	{"type":"integer"}	—
TYPE_FLOAT / TYPE_DOUBLE	{"type":"number"}	—
TYPE_BOOL	{"type":"boolean"}	—
TYPE_ENUM	{"type":"string","enum":[...]}	Values from EnumDescriptor
TYPE_MESSAGE (nested)	{"$ref":"#/$defs/TypeName"}	Recursive; placed in $defs once
repeated field	{"type":"array","items":{...}}	—
map<K, V>	{"type":"object","additionalProperties":{V}}	Key type constraint dropped; noted in description
oneof	{"oneOf":[...]}	Near-equivalent; description notes mutual exclusion

4.2 Core algorithm (pseudocode)

```
json Convert(const Descriptor* msg, json& defs) {
    if (defs.contains(msg->full_name()))
        return {"$ref": "#/$defs/" + msg->full_name()};

    defs[msg->full_name()] = nullptr;  // reserve slot; breaks cycles

    json schema = {"type": "object", "properties": {}};
    for (auto* f : msg->fields()) {
        json fs = f->is_map()      ? ConvertMap(f, defs)
                : f->is_repeated() ? {"type":"array","items":ConvertField(f,defs)}
                :                    ConvertField(f, defs);
        fs["description"] = GetComment(f);
        schema["properties"][f->json_name()] = fs;
    }
    for (auto* oneof : msg->oneofs()) {
        json variants;
        for (auto* f : oneof->fields()) variants.push_back(ConvertField(f, defs));
        schema["oneOf"].push_back(variants);
    }
    defs[msg->full_name()] = schema;
```

```
    return {"$ref": "#/$defs/" + msg->full_name()};
}
```

> *Cycle detection: the slot is reserved before recursing. Any field that later references the same type finds the placeholder and emits a $ref instead of recursing infinitely.*

5. ToolRegistry & Tool Naming

5.1 ToolEntry structure

```
struct ToolEntry {
    std::string              name;        // unique across all services
    std::string              description; // from proto leading comment
    const MethodDescriptor*  method;      // owned by descriptor pool
    mutable LazyValue<json>  schema;      // computed on first tools/list

    const json& GetInputSchema() const {
        return schema.GetOrInit([this]() {
            json defs;
            auto s = ProtoSchemaConverter::Convert(method->input_type(), defs);
            s["$defs"] = std::move(defs);
            return s;
        });
    }
};
```

5.2 Tool name generation

Names follow ServiceName_MethodName (e.g., UserService_GetUser). For names exceeding 48 characters, a truncated + hash scheme prevents LLM token-limit issues while preserving uniqueness:

```
std::string MakeToolName(const MethodDescriptor* m) {
    std::string full = m->service()->name() + "_" + m->name();
    if (full.size() <= 48) return full;
    return full.substr(0, 32) + "_" + Hex(Murmur3(full), 8);
}
```

The full method path (package.Service/Method) is always injected into the description header, allowing LLMs and humans to reverse-look up the origin method.

5.3 Description fallback

```
// When no leading proto comment is found:
"[RPC] package.UserService/GetUser — input: GetUserRequest, output: GetUserResponse"
```

> *Best practice: always write proto comments. The LLM selects the correct tool almost entirely based on the quality of this description.*

6. Streaming RPC Handling

MCP tools/call is a request-response model. Server-streaming RPCs require an explicit bridging strategy.

6.1 Decision matrix

RPC type	Registration	Call behaviour
Unary	Always registered	Direct pass-through
Server-streaming (small)	Registered; "[streaming]" in description	Responses aggregated into a JSON array; returned as single result
Server-streaming (large)	Registered; "[streaming]" in description	Returns error: "response exceeds limit; use subscribe_X instead"
Client-streaming / Bidi	Skipped by default	Configurable: skip or aggregate-and-close

6.2 Aggregation threshold

```
struct StreamingConfig {
    size_t aggregate_max_bytes       = 1 * 1024 * 1024;  // 1 MB default
    bool   register_client_streaming = false;
    bool   register_bidi_streaming   = false;
};
```

The threshold must not be hardcoded. It is injected at MCPSession construction time so operators can tune it per deployment without recompilation.

7. Error Handling

7.1 Error taxonomy

Source	JSON-RPC code	Strategy
Unknown tool name	-32601 MethodNotFound	Return list of registered names for LLM self-correction
JSON → proto deserialization failure	-32602 InvalidParams	Include field path and expected type
RPC invocation failure	-32603 InternalError	Propagate gRPC status code; strip internal stack traces
Streaming response too large	-32603 InternalError	Explain limit and suggest alternative tool
Proto schema conversion error	N/A — startup only	Fail-fast at registration; do not serve partial registry

7.2 Unknown-field tolerance

- Set . LLMs occasionally hallucinate extra keys; silently dropping them is preferable to hard-failing a call.
- Log a with the tool name and the list of dropped fields for post-hoc debugging.

8. Performance Design Notes

8.1 Hot path optimizations

Optimization	Location	Impact
Zero-copy JSON parsing	ArgumentsDeserializer	Eliminates one string allocation per call
Lazy schema generation	ToolEntry::GetInputSchema()	Schema cost paid at most once; zero cost for never-listed tools
Arena allocator on response	RpcInvoker::Invoke()	Reduces heap fragmentation on high-QPS services
Immutable ToolRegistry	ToolRegistry	Lock-free reads after startup — no mutex on list or call
$ref deduplication in schema	ProtoSchemaConverter	Prevents exponential JSON size for deeply shared message types

8.2 Estimated steady-state bridge overhead

For a single tools/call round-trip (excluding network latency and RPC handler execution time):

- JSON parse (ArgumentsDeserializer): ~5–20 μs depending on payload size.
- Proto → JSON serialization of response: ~5–20 μs.
- Total bridge overhead target: < 50 μs p99.

9. Implementation Plan

9.1 Component size estimates

Component	Est. lines	Key dependencies
LazyValue helper	~30	std::call_once
ToolEntry + ToolRegistry	~80	absl::flat_hash_map
ProtoSchemaConverter	~350	protobuf descriptor API, nlohmann/json
ArgumentsDeserializer	~40	protobuf JsonStringToMessage util
RpcInvoker interface	~20	absl::Status, std::future
GrpcInvoker (impl)	~100	grpc++
MCPSession (state machine)	~200	nlohmann/json, RpcInvoker
Streaming aggregation logic	~60	RpcInvoker, StreamingConfig
Total	~880	

9.2 Recommended build order

1. LazyValue + ToolEntry — no external deps; fully unit-testable in isolation.
2. ProtoSchemaConverter — test against hand-crafted .proto files; verify $ref deduplication and cycle handling.
3. RpcInvoker interface + InProcInvoker mock — enables end-to-end tests without a real gRPC stack.
4. ArgumentsDeserializer — fuzz with malformed JSON; verify unknown-field handling and zero-copy path.
5. MCPSession state machine — test full JSON-RPC 2.0 lifecycle (initialize, list, call, error paths).
6. GrpcInvoker — integration tests against a real gRPC server; verify arena allocator.
7. Streaming aggregation — test with payloads above and below threshold; verify error message on overflow.

10. Known Limitations

Limitation	Severity	Mitigation
proto3 has no required fields — all are optional in generated schema	Low	Add custom FieldOption for mandatory fields; Converter reads it
bytes fields lose semantic meaning (base64 in JSON Schema)	Medium	Annotate field description with expected binary format
map<K,V> key type constraint dropped	Low	Document in field description; key is always string at JSON layer
oneof approximated with oneOf (not fully equivalent)	Low	Add description noting mutual exclusion
Dynamic proto loading / hot-reload not supported	Low	Restart required on schema change
No per-tool authorization at bridge level	Medium	Delegate to RPC layer (gRPC interceptors / middleware)

11. Extension Points

11.1 Custom FieldOptions

The Converter checks for custom proto options before falling back to auto-generation. This allows teams to inject business-level semantics without modifying the bridge:

```
extend google.protobuf.FieldOptions {
    string description = 50001;  // Override auto-generated field description
    string example     = 50002;  // JSON example value for the schema
    bool   required    = 50003;  // Mark field as required in JSON Schema
    string format      = 50004;  // JSON Schema "format" (e.g. "email", "uri")
}
```

11.2 Pluggable RpcInvoker

Any transport backend can be used by implementing the RpcInvoker interface. The MCP adapter binary can target gRPC, in-process handlers, Thrift, or a custom transport without recompilation of the adapter itself. This also enables InProcInvoker for unit testing with no network overhead.

11.3 Schema post-processing hook

An optional SchemaPostProcessor callback can be registered to augment or override generated schemas — useful for injecting value ranges, pattern restrictions, or conditional requirements that proto has no native expression for:

```
using SchemaPostProcessor =
    std::function<void(std::string_view field_full_name, json& field_schema)>;

// Example: enforce age > 0
converter.RegisterPostProcessor("user.User.age",
    [](std::string_view, json& s) {
        s["minimum"] = 1;
        s["maximum"] = 150;
    });
```